edinpub.in

Introduction to AXI-Lite

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

Introduction to AXI-Lite — Valid/Ready Handshake and Channel Architecture

2. Objective

By the end of this module, learners will be able to:

- Describe the five AXI-Lite channel types and their direction
- Explain the valid/ready handshake mechanism and its rules
- Trace a complete write transaction (AW → W → B) through waveforms
- Trace a complete read transaction (AR → R) through waveforms
- Run the standalone AXI-Lite testbench (`tb_axi_lite_standalone.v`) and verify the handshake

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

Imagine AXI-Lite as a two-person handshake:

- The **master** (e.g., a CPU) says "I have something for you" by raising `VALID`
- The **slave** (e.g., a peripheral) says "I'm ready to receive" by raising `READY`
- The actual transfer (handshake) only happens **when BOTH are high at the same clock edge**

This means the master and slave can independently assert their signals without knowing each other's timing — a protocol designed for modular, decoupled hardware blocks.

Write operation analogy (posting a letter):

1. You (master) write the address on the envelope → **AW channel**
2. You put the letter in the envelope (data) → **W channel**
3. The post office confirms delivery → **B channel (response)**

Read operation analogy (fetching a file):

1. You request a file by name → **AR channel**
2. The librarian hands you the file → **R channel**

3.2 Intermediate — Engineering View

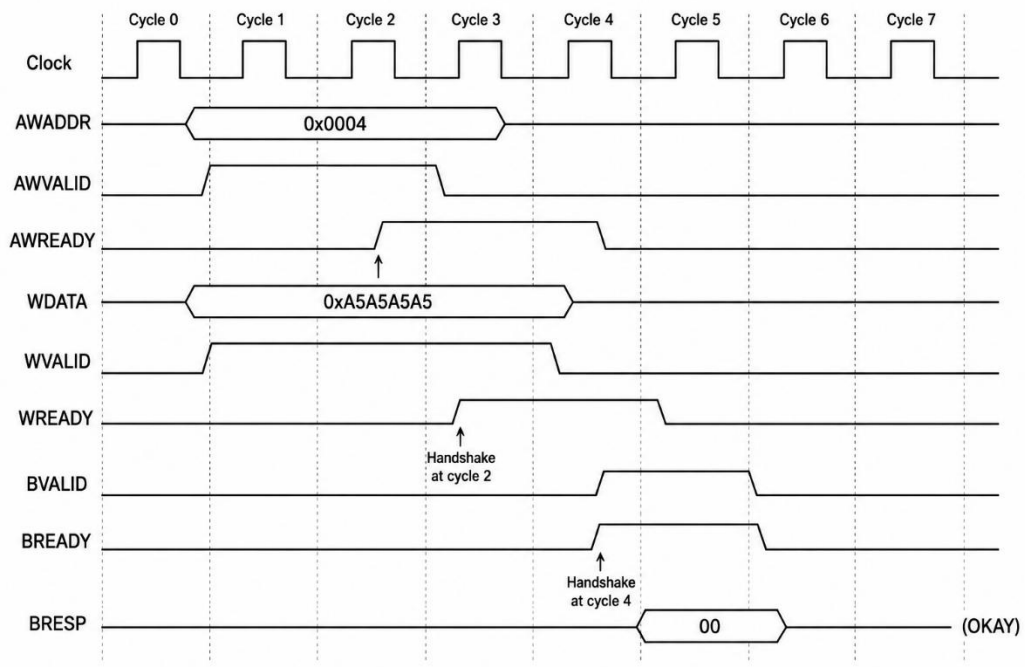
AXI4-Lite is a simplified subset of ARM's AMBA AXI4 protocol, designed for register-level peripheral access. It has:

Channel	Direction	Signals
Write Address (AW)	Master → Slave	`AWADDR[31:0]`, `AWVALID`, `AWREADY`
Write Data (W)	Master → Slave	`WDATA[31:0]`, `WSTRB[3:0]`, `WVALID`, `WREADY`
Write Response (B)	Slave → Master	`BRESP[1:0]`, `BVALID`, `BREADY`
Read Address (AR)	Master → Slave	`ARADDR[31:0]`, `ARVALID`, `ARREADY`
Read Data (R)	Slave → Master	`RDATA[31:0]`, `RRESP[1:0]`, `RVALID`, `RREADY`

Handshake Golden Rule: > A transaction occurs at the rising clock edge when **VALID AND READY** are both 1.

Key constraints: - Master can assert VALID without knowing if slave is ready - Slave can assert READY without VALID being high (pre-ready) - Master MUST NOT de-assert VALID before a handshake completes - BRESP/RRESP = `2'b00` means OKAY (no error) - WSTRB is a **byte-enable**: bit 0 controls bytes [7:0], bit 1 → [15:8], etc.

3.3 Advanced — Signal/Architecture Level



Why separate AW and W channels? This decoupling allows a slave to accept an address before it is ready to accept data, and vice versa — enabling pipelining and burst (in AXI4 full) optimization.

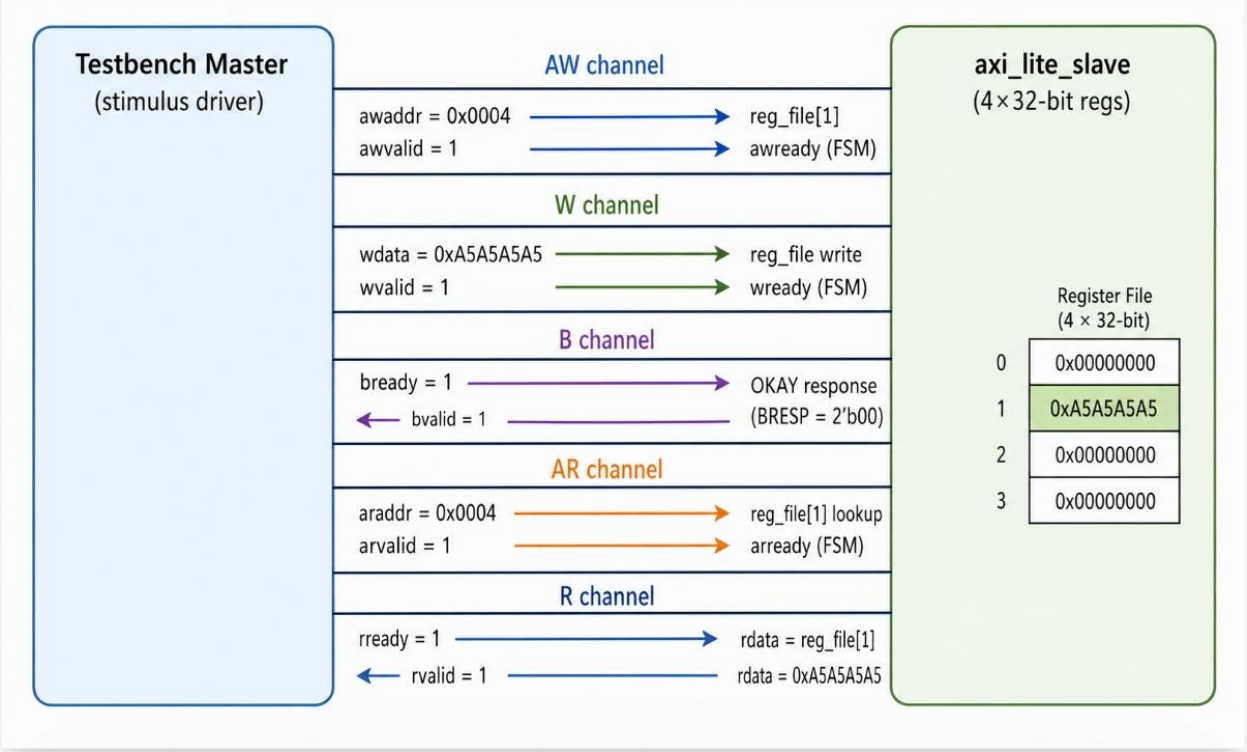
WSTRB encoding:

```
WSTRB = 4'hF  → Write all 4 bytes (common case, 32-bit register write)
WSTRB = 4'h1  → Write byte [7:0]   only
WSTRB = 4'h3  → Write bytes [15:0] only
WSTRB = 4'h0  → No write (bypass)
```

4. Architecture / Concept

Standalone AXI-Lite Educational Model

The testbench `tb/tb_axi_lite_standalone.v` implements a complete self-contained demonstration:



AXI-Lite Slave State Machines (from `tb_axi_lite_standalone.v`)

Write FSM:

```
IDLE → (awvalid asserted) → WR_ADDR (latch addr)
WR_ADDR → (wvalid asserted) → WR_DATA (latch data, write to reg)
```

```
WR_DATA → (always) → WR_RESP (assert bvalid)
WR_RESP → (bvalid & bready) → IDLE
```

Read FSM:

```
IDLE → (arvalid asserted) → RD_ADDR (latch addr, pre-load rdata)
RD_ADDR → (always) → RD_DATA (assert rvalid)
RD_DATA → (rvalid & rready) → IDLE
```

5. Implementation

Files in This Module

File	Role
`tb/tb_axi_lite_standalone.v`	Self-contained AXI-Lite slave + master stimulus testbench
`tb/tb_axi_lite.v`	Full AXI-Lite testbench with UART peripheral
`tb/tb_axi_lite_master.v`	Reusable AXI-Lite master task library
`rtl/axi_lite_interconnect.v`	Production AXI-Lite interconnect (1M → 3S)

Standalone Testbench Key Tasks (`tb_axi_lite_standalone.v`)

Write Task (lines 273–318):

```
task axi_write;
    input [ADDR_WIDTH-1:0] addr;
    input [DATA_WIDTH-1:0] data;
    // Phase 1: AW channel – present address, wait for awready
    // Phase 2: W channel  – present data, wait for wready
    // Phase 3: B channel  – wait for bvalid, de-assert bready
endtask
```

Read Task (lines 321–351):

```
task axi_read;
    input  [ADDR_WIDTH-1:0] addr;
    output [DATA_WIDTH-1:0] rd_data;
    // Phase 1: AR channel – present address, wait for arready
    // Phase 2: R channel  – wait for rvalid, latch rdata
endtask
```

Slave Register Map (`axi_lite_slave` in standalone TB)

Address	Index	Description
`0x0000`	reg_file[0]	Register 0
`0x0004`	reg_file[1]	Register 1 (used in lab)
`0x0008`	reg_file[2]	Register 2
`0x000C`	reg_file[3]	Register 3

Address-to-index formula: `index = (addr >> 2) % NUM_REGS`

6. Lab / Hands-on Section

Step 1: Run the Standalone AXI-Lite Testbench

`cd /home/coe-9/OpenLaneUser/designs/picorv32_soc/`

Project Path: /home/coe-9/OpenLaneUser/designs/picorv32_soc/
This project is configured under the user directory coe-9.
Users accessing the project on a different account must update the username in all paths, scripts, and environment configurations accordingly.
Example:
`/home/coe-9/ → /home/<your_username>/`

```
make axi_sa
```

This command: 1. Cleans `obj_axi_sa/` 2. Verilates `tb/tb_axi_lite_standalone.v` with `--binary --trace --timing` 3. Runs `./obj_axi_sa/sim_axi_sa` → produces `tb_axi_lite_standalone.vcd` 4. Opens GTKWave

Expected Terminal Output

```
=========================================================
  AXI-Lite Standalone Testbench  (32-bit addr/data)
=========================================================

[TB] Reset applied → All signals initialized

[TB] Starting AXI-Lite Write Transaction...

[WRITE] Address = 0x00000004
[WRITE] Data    = 0xa5a5a5a5
[WRITE] awvalid=1, wvalid=0 → Sending address first...
[WRITE] awready=1 → Handshake Done (address 0x00000004 accepted)
[WRITE] wvalid=1 → Sending data 0xa5a5a5a5...
[WRITE] wready=1 → Handshake Done (data accepted)
[WRITE] Waiting for bvalid...
[WRITE] bvalid=1 → Write Response Received (OKAY)

[TB] Starting AXI-Lite Read Transaction...

[READ] Address = 0x00000004
[READ] arvalid=1 → Waiting for arready...
[READ] arready=1 → Handshake Done (address 0x00000004 accepted)
[READ] Waiting for rvalid...
[READ] rvalid=1 → Data Received = 0xa5a5a5a5

[CHECK] Write Data == Read Data → PASS ✓
```

```
[TB] AXI-Lite Transaction Successful

=======================================================
  SUMMARY :  Passed = 1   Failed = 0
=======================================================
[TB] Simulation Finished
```

Step 2: View Waveforms

```
gtkwave tb_axi_lite_standalone.vcd &
```

Add signals to the wave window in order:

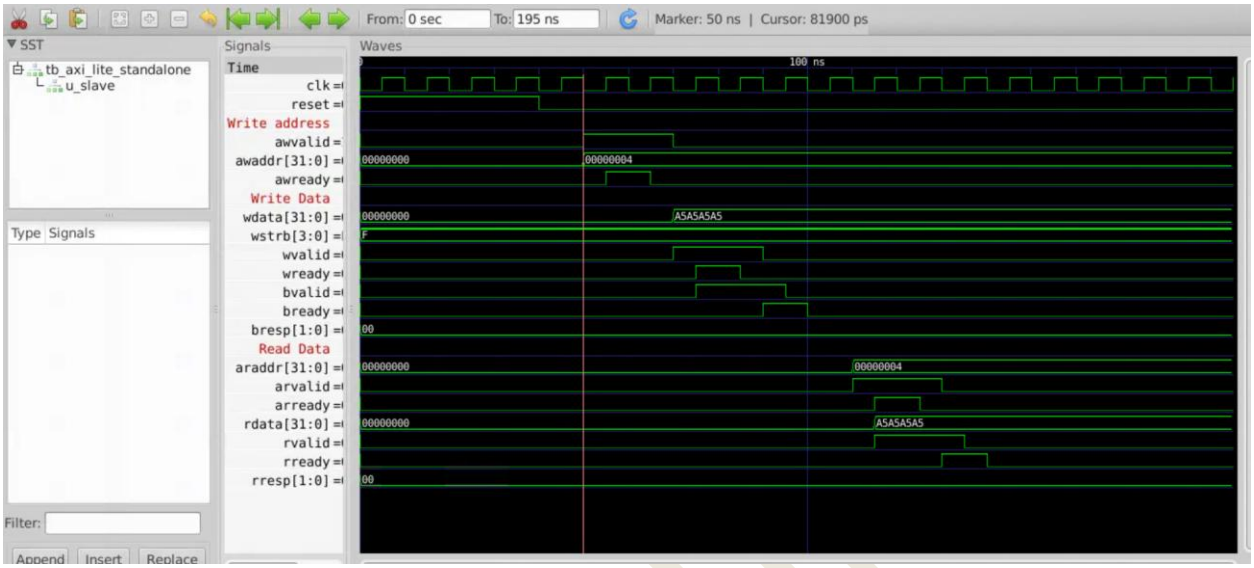
```
clk
reset
awaddr[31:0]
awvalid
awready
wdata[31:0]
wstrb[3:0]
wvalid
wready
bvalid
bready
bresp[1:0]
araddr[31:0]
arvalid
arready
rdata[31:0]
rvalid
rready
rresp[1:0]
```

Step 3: Run Full AXI-UART Testbench

```
make axi_uart
```

echiphub.in

7. Waveform / Verification Insight



Write Transaction in GTKWave

Look for the pattern:

1. `awvalid` rises → address appears on `awaddr` → wait for `awready` to rise → **AW handshake**
2. `wvalid` rises → data appears on `wdata` → wait for `wready` to rise → **W handshake**
3. `bvalid` rises → `bready` asserted → **B handshake** (OKAY response)
4. All channels de-assert → transaction complete

Read Transaction in GTKWave

1. `arvalid` rises → address on `araddr` → `arready` rises → **AR handshake**
2. `rvalid` rises → `rdata` has the value → `rready` asserted → **R handshake**
3. Verify `rdata == wdata` (0xA5A5A5A5) → data integrity confirmed

Timing Measurement

In GTKWave, use markers to measure: - **AW handshake latency:** Cycles from `awvalid=1` to `awready=1` - **Write-to-response latency:** Cycles from W handshake to B handshake - **Read latency:** Cycles from AR handshake to R handshake

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
`BVALID` never asserts	AW or W handshake never completed	Check both `awvalid/awready` AND `wvalid/wready` completed
`RVALID` never asserts	AR handshake deadlock	Ensure `arvalid` held until `aready` seen
Data mismatch on read	Write used wrong `WSTRB`	Verify `WSTRB = 4'hF` for full 32-bit write
Bus hangs permanently	Master de-asserted VALID before handshake	Violation of AXI rule: "never de-assert VALID before READY"
Watchdog fires (500 µs)	Handshake deadlock	Add `$display` inside FSM states to trace which state stuck
Multiple writes overlap	Missing B-channel completion	CPU must wait for BVALID before starting next write

AXI Protocol Violation Checklist

- ☐ `AWVALID held until AWREADY seen?`
- ☐ `WVALID held until WREADY seen?`
- ☐ `BREADY asserted before or when BVALID was seen?`
- ☐ `ARVALID held until ARREADY seen?`
- ☐ `RREADY asserted before or when RVALID seen?`
- ☐ `No simultaneous multi-master (AXI-Lite = single master)?`

9. Summary

Concept	Key Point
5 channels	AW, W, B (write side) + AR, R (read side)
Handshake rule	Transfer occurs when VALID **AND** READY = 1 at clock edge
WSTRB	Byte-enable mask; 4'hF = full 32-bit write
BRESP/RRESP	2'b00 = OKAY; no error in this design
Write sequence	AW handshake → W handshake → B response
Read sequence	AR handshake → R data handshake
Lab result	Write 0xA5A5A5A5 → reg[1]; Read back → 0xA5A5A5A5; PASS ✓

Previous: *Sub-module 1 — RISC-V ISA*
Next: *Sub-module 3 — UART Basics*